Arogya-Nidhi

Health Scheme Eligibility Checker

Android App Development using GenAI

Project Requirements Document

Version 1.0 | Internship Project | 2026

Amulya A

Questers G5 | Project #61

Platform: Android | Language: Kotlin | Domain: Healthcare | GenAI Powered

! 1. Problem Statement

> **Core Problem:** Millions of rural families in India remain unaware of government health schemes they are legally entitled to. Despite the existence of schemes like Ayushman Bharat, ESIC, and state-level Arogyasri programmes, the 'Information Barrier' causes families to pay out-of-pocket for surgeries and treatments that should have been free.

Key Pain Points

Pain Point	Description
Information Gap	Rural citizens lack awareness of which schemes apply to them.
Document Confusion	Families don't know which documents to carry, leading to failed applications.
Financial Burden	Households fall into debt or skip treatment due to assumed unaffordability.
Geographic Barrier	No easy way to find nearby empaneled hospitals that accept these schemes.
Language Barrier	Government portals are often in English or formal Hindi, inaccessible to many.

★ 2. Project Vision & Description

Arogya-Nidhi (meaning *Health Wealth*) is a digital counselor — an Android application powered by GenAI that acts as a bridge between India's health scheme ecosystem and its most vulnerable citizens. By answering a simple 5-question quiz, any family can discover exactly which government schemes they qualify for, what documents they need, and which hospitals near them accept those schemes.

Vision Pillars

Accessibility: Works on low-end Android phones; minimal data usage.

Simplicity: Stepper UI with plain-language questions — no literacy barrier.

Empathy: Warm, encouraging tone designed for first-time users.

Accuracy: Decision-tree logic maps family profile to eligible schemes precisely.

Actionability: Does not just inform — hands the user a document checklist to act immediately.

■ 3. Scope of the Application

In Scope

✓ Eligibility quiz (5 structured questions) covering income, BPL status, occupation, family size, and disability.

✓ Matching engine: maps user profile to a curated database of central and state health schemes.

✓ Per-scheme document checklist stored locally (JSON / Room DB).

✓ Empaneled hospital finder with district-level search and map integration.

✓ GenAI-powered chat assistant for follow-up questions in simple language.

✓ Offline-first design: core eligibility logic works without internet.

✓ Multi-language support (Hindi + regional language for MVP).

Out of Scope (v1.0)

✗ Direct online scheme application or form submission (government portal integration).

✗ Real-time scheme status tracking or claim processing.

✗ Telemedicine or doctor-consultation features.

✗ Biometric or Aadhaar authentication.

✗ Coverage of private health insurance products.

■ 4. App Usage & User Flow

Flow Step	Detail
Step 1 — Onboarding	User opens app; greeted with empathetic welcome screen. Language selection prompt.
Step 2 — Eligibility Quiz	Stepper UI walks user through 5 questions: 1. Annual household income 2. BPL card holder? (Yes/No) 3. Occupation category (Farmer / Daily Wage / Govt. Employee / Other) 4. Family size 5. Any member with disability?
Step 3 — Scheme Results	AI-powered engine returns: 'You are eligible for [Scheme A] and [Scheme B].' Each scheme card shows benefit amount and coverage details.
Step 4 — Document Guide	Tap any scheme card to see a tailored document checklist (e.g., 'Aadhaar Card, BPL Card, Income Certificate').
Step 5 — Hospital Finder	Searchable list of empaneled hospitals filtered by District. Tap to open in Maps.
Step 6 — GenAI Assistant	FloatingActionButton opens a chat — user can ask follow-up questions in natural language.

F 5. Functional Requirements

Req. ID	Requirement Description
FR-01	The app shall display a 5-step Stepper UI for the eligibility quiz.
FR-02	The system shall apply decision-tree logic to map user inputs to eligible schemes.
FR-03	Scheme results shall update dynamically when income or BPL status changes.
FR-04	Each scheme shall have an associated document checklist stored in Room DB / JSON.
FR-05	The hospital list shall be searchable and filterable by District name.
FR-06	The app shall function in offline mode for the core eligibility and checklist features.
FR-07	The GenAI assistant shall respond to user queries in plain, simple language.
FR-08	The app shall support at least two languages (Hindi + one regional language).
FR-09	Users shall be able to share/export their scheme eligibility result as a PDF or image.

FR-10	The app shall display the nearest empaneled hospital using device GPS when permission is granted.

N 6. Non-Functional Requirements

ID	Category	Requirement
NFR-01	Performance	App launch to quiz screen in under 2 seconds on a mid-range device.
NFR-02	Usability	UI must pass accessibility checks; minimum font size 14sp; high-contrast mode.
NFR-03	Reliability	Core eligibility logic must return correct results for 100% of test cases defined in success criteria.
NFR-04	Scalability	Scheme database must be updatable via JSON config file without app redeployment.
NFR-05	Security	No personally identifiable data shall be transmitted to external servers without explicit consent.
NFR-06	Compatibility	App shall support Android 8.0 (API 26) and above.
NFR-07	Availability	Core features (quiz + checklist) must be available 100% offline.
NFR-08	Maintainability	Code must follow MVVM architecture; minimum 60% unit test coverage on decision-tree logic.

★ 7. Feature List

Feature	Description	Priority
5-Step Eligibility Quiz	Stepper UI with income, BPL, occupation, family size, disability questions.	**Must-Have**
Scheme Matching Engine	Decision-tree / AI logic that maps user profile to eligible schemes accurately.	**Must-Have**
Scheme Result Screen	Displays matched scheme cards with benefit summary and coverage details.	**Must-Have**
Document Checklist	Per-scheme checklist stored in Room DB; viewable and shareable offline.	**Must-Have**
Empaneled Hospital List	Searchable by District; shows address and contact details.	**Must-Have**
Offline Core Functionality	Quiz, results, and checklist work without internet connection.	**Must-Have**
Multi-language Support	At least Hindi + one regional language for the target demographic.	**Must-Have**
Accessible & Empathetic UI	High-contrast, large fonts, plain language, warm visual design.	**Must-Have**
GenAI Chat Assistant	Floating chat button powered by a language model for natural-language Q&A.	**Good-to-Have**

GPS-Based Hospital Finder	Auto-detect user location and show nearest empaneled hospitals on map.	**Good-to-Have**
Result Export / Share	Export eligibility result + document checklist as PDF or shareable image.	**Good-to-Have**
Push Notifications	Remind users about upcoming scheme deadlines or document expiry.	**Good-to-Have**
Scheme Application Tracker	Simple manual tracker where users note their application status.	**Good-to-Have**
Voice Input	Allow users to answer quiz questions using voice for low-literacy users.	**Good-to-Have**
Dark Mode	Battery-saving dark theme for AMOLED devices common in rural areas.	**Good-to-Have**
Admin CMS Panel	Web-based panel to update schemes and hospitals without app update.	**Good-to-Have**

■ 8. Technical Implementation

Component	Details
Architecture	MVVM (Model-View-ViewModel) with Repository pattern for clean separation of concerns.
Language	Kotlin (primary); XML layouts with Material Design 3 components.
UI Components	Stepper library for quiz flow; RecyclerView for hospital list; CardView for scheme results.
Decision Tree	Kotlin when-expression / sealed class state machine mapping user inputs to scheme eligibility rules.
Local Storage	Room DB for scheme data + document checklists; SharedPreferences for user session.
GenAI Integration	Google Gemini API (or equivalent) via Retrofit for the chat assistant feature.
Networking	Retrofit + OkHttp for API calls; Gson for JSON parsing.
Maps & Location	Google Maps SDK + FusedLocationProviderClient for hospital finder.
Dependency Injection	Hilt (Dagger-Hilt) for clean DI across ViewModels and Repositories.
Testing	JUnit 4 + Mockito for unit tests on decision-tree logic; Espresso for UI tests.
Build & CI	Gradle build system; ProGuard for release build obfuscation.

❤ 9. Impact Goals

> **Universal Health Coverage**
> Ensure the poorest citizens benefit from government safety nets by removing the information barrier.

Social Justice
Democratize access to health entitlements — no middlemen, no bribery, no confusion.

Financial Protection
Prevent medical debt by connecting families with free-treatment cards before they need hospitalisation.

Health Literacy
GenAI assistant educates users about their rights in plain language, building lasting awareness.

Policy Feedback Loop
Anonymised usage analytics (opt-in) can help policymakers identify under-served regions.

■ 10. 3-Week Delivery Timeline

WEEK 1 | DAYS 1–7	**Setup + Core Logic**	→ Set up Android Studio + all dependencies (Hilt, Room, Retrofit, Gemini API) → Design all screens in Figma / XML layouts → Build 5-step eligibility quiz Stepper UI → Implement decision-tree scheme matching engine → Set up Room DB with scheme data and document checklists → Checkpoint: User can complete quiz and see scheme results
WEEK 2 | DAYS 8–14	**Features + Integration**	→ Build scheme result screen with cards and benefit summary → Implement per-scheme document checklist screen → Add empaneled hospital list with district-level search → Integrate GPS-based hospital finder (FusedLocationProviderClient) → Build GenAI chat assistant screen with Gemini API → Checkpoint: Full user flow works end-to-end
WEEK 3 | DAYS 15–21	**Polish + Delivery**	→ Add multi-language support (Hindi + one regional language) → Polish UI — accessibility, contrast, font sizes, empathetic tone → Add app icon, splash screen, and result export / share feature → Performance test on a real device with full scheme dataset → Write README + capture screenshots + build signed APK → Push to GitHub + submit for review

✓ 11. Success Criteria for Students

Criteria ID	Success Criterion
SC-01	The eligibility result must change accurately when income level or BPL status is modified — zero incorrect mappings across all test cases.

SC-02	The hospital list must be searchable by District with results appearing in under 300ms.
SC-03	The UI must be assessed as 'empathetic and simple' by at least 3 of 5 rural-persona user testers.
SC-04	Document checklist must be accurate for at least 5 distinct central/state government schemes.
SC-05	The GenAI assistant must correctly answer at least 80% of pre-defined test queries about scheme eligibility.
SC-06	App must function fully offline for quiz, result, and document checklist flows.
SC-07	Code review must confirm MVVM architecture adherence and minimum 60% unit test coverage on decision-tree logic.